Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО



Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа № 2
«Последовательный интерфейс UART»

по дисциплине

«Проектирование вычислительных систем»

Вариант № 5: «Кодовый замок»

Выполнили: Ястребов-Амирханов Алекси, Исупов Никита Александрович
Группа: P3432/P3431
Преподаватель: Ключев Владислав Аркадьевич

Санкт-Петербург,

2026

Оглавление

1. Цели работы 3
2. Описание варианта задания 3
2.1. Общие требования к подсистеме UART 3
2.2. Требования к прикладной задаче «Кодовый замок» (Вариант 5) 4
3. Аппаратная платформа и схема подключения стенда SDK-1.1M 4
4. Архитектурное описание программы и алгоритмов 6
4.1. Диспетчеризация и принципы кооперативной многозадачности 6
4.2. Архитектура драйвера UART и реализация кольцевого буфера FIFO 7
4.3. Режимы работы драйвера UART: опрос и прерывания 8
4.4. Логика конечного автомата кодового замка 9
5. Анализ регистров USART в микроконтроллерах STM32F4 11
5.1. Архитектура и назначение регистров периферийного блока USART 11
5.2. Анализ механизмов обработки прерываний и сброса флагов 12
6. Методика и результаты экспериментальной верификации 12
7. Выводы 14

1. Цели работы

1. Изучить протокол последовательной асинхронной передачи данных UART (Universal Asynchronous Receiver-Transmitter), форматы информационных посылок и принципы сопряжения микроконтроллеров с персональным компьютером на примере стенда SDK-1.1M.

2. Изучить структуру и принципы функционирования подсистемы прерываний в 32-разрядных микроконтроллерах семейства STM32F4 на базе аппаратного контроллера векторных прерываний NVIC (Nested Vectored Interrupt Controller) ядра ARM Cortex-M4.

3. Освоить аппаратное устройство периферийных контроллеров USART микроконтроллера `STM32F427VIT6`, изучить назначение и функции управляющих и статусных регистров (`SR`, `DR`, `BRR`, `CR1`, `CR2`, `CR3`).

4. Спроектировать и реализовать двухрежимный драйвер интерфейса UART: в режиме неблокирующего циклического опроса аппаратных флагов (polling) и в режиме обслуживания векторных прерываний (interrupt) с программной буферизацией входящего потока данных в кольцевом FIFO-буфере.

5. Модернизировать прикладную программу «Кодовый замок» (Вариант 5) для ввода отпирающей последовательности через последовательный канал связи на скорости 9600 бит/с с поддержкой эхо-вывода, динамического изменения секретного пароля и переключения между режимами опроса и прерываний по нажатию аппаратной кнопки стенда.

2. Описание варианта задания

2.1. Общие требования к подсистеме UART

В рамках лабораторной работы требуется разработать два взаимозаменяемых режима функционирования последовательного интерфейса UART для стенда SDK-1.1M:

- **Режим без прерываний (Polling):** прием символов должен производиться без применения аппаратных прерываний от блока UART. Функция приема обязана быть строго неблокирующей — при отсутствии поступившего байта в регистре данных функция немедленно возвращает управление в вызывающий контекст, исключая зависание и простой вычислителя в цикле ожидания.
- **Режим с использованием прерываний (Interrupt):** прием байтов выполняется в асинхронном фоне специализированным обработчиком прерывания по флагу `RXNE`. Драйвер обязан реализовать промежуточный буфер типа FIFO (First-In, First-Out) фиксированной длины для гарантированного предотвращения потери данных в моменты занятости основного цикла программы. Вызов функции чтения извлекает символ из буфера без ожидания. В данном режиме глобальное прерывание от контроллера UART должно быть предварительно разрешено в блоке NVIC и в регистрах самого интерфейса.
- **Эхо-вывод:** каждый принимаемый микроконтроллером символ должен немедленно транслироваться обратно в последовательный канал связи для отображения в терминале пользователя. Каждое информационное сообщение стенда должно выводиться с новой строки (последовательность символов возврата каретки и перевода строки).
- **Управление режимами работы:** переключение между режимом опроса и режимом прерываний должно выполняться по нажатию тактовой кнопки боковой панели стенда (вывод `PC15`). Каждое переключение обязано сопровождаться отправкой в терминал текстового уведомления о текущем активированном режиме. При смене режима

работы система должна сбрасывать все ранее введенные пользователем данные замка и выводить приглашение к повторному вводу.

2.2. Требования к прикладной задаче «Кодовый замок» (Вариант 5)

Прикладная логика кодового замка адаптируется для взаимодействия через интерфейс UART на скорости 9600 бит/с:

- **Ввод кода:** ввод отпирающей комбинации выполняется символами латинского алфавита и цифрами через терминал. Сравнение символов производится без учета регистра (регистронезависимо). Длина пароля по умолчанию составляет 8 символов.
- **Индикация элементов кода:** каждый правильно введенный символ сопровождается короткой вспышкой желтого светодиода (200 мс). Ошибочный символ сопровождается вспышкой красного светодиода (300 мс), сбросом текущей позиции ввода в начало и инкрементом счетчика ошибок.
- **Открытие замка:** при вводе полной правильной 8-символьной цепочки на 3 секунды зажигается зеленый светодиод, в терминал выводится строка подтверждения доступа, а ввод новых символов на время открытия блокируется. По истечении 3 секунд зеленый светодиод гаснет, и замок возвращается в исходное состояние.
- **Режим тревоги:** после трех ошибочных попыток ввода подряд активируется блокировка системы на 5 секунд. В течение этого времени непрерывно мигает красный светодиод, терминал выводит предупреждение о блокировке, а любые поступающие символы игнорируются.
- **Таймаут ввода:** если ввод был начат, но в течение 5 секунд следующий символ не поступил, позиция ввода сбрасывается в начало, красный светодиод выдает вспышку 300 мс, а в терминал отправляется сообщение об истечении времени ожидания.
- **Изменение пароля:** по вводу специального символа + система переходит в режим смены пароля:
    1. Пользователь вводит новую комбинацию длиной до 8 символов; завершение ввода происходит либо по нажатию клавиши Enter, либо автоматически по достижении 8 символов;
    2. Стенд отправляет запрос на подтверждение активации нового кода;
    3. При получении символа y или Y новый пароль становится активным; при вводе любого другого символа процедура смены отменяется, и сохраняется старый пароль.

3. Аппаратная платформа и схема подключения стенда SDK-1.1M

Лабораторная работа выполняется на базе аппаратного стенда-конструктора SDK-1.1M с процессорным модулем на базе микроконтроллера `STM32F427VIT6`.

В стенде SDK-1.1M аппаратный интерфейс взаимодействия с инструментальным компьютером реализован на базе микросхемы преобразователя USB-UART фирмы FTDI, подключенной к микроконтроллеру через линии блока `USART6`. Преобразователь создает в операционной системе виртуальный COM-порт, через который осуществляется обмен данными по общему кабелю программирования и отладки USB Micro-B:

Таблица 1 — Аппаратные ресурсы стенда SDK-1.1M, используемые в работе

Функциональный узел	Контакт MCU	Конфигурация вывода	Направление сигнала	Схемотехническое назначение
Интерфейс USART6 (TX)	PC6	Альтернативная функция AF8 (USART6_TX)	Выход MCU	Линия передачи данных из микроконтроллера в FTDI (виртуальный COM-порт компьютера).
Интерфейс USART6 (RX)	PC7	Альтернативная функция AF8 (USART6_RX)	Вход MCU	Линия приема данных микроконтроллером от терминала компьютера.
Кнопка боковой панели	PC15	Цифровой вход с подтяжкой Pull-Up	Вход MCU	Тактовая кнопка SB1 (активный уровень 0). Служит для переключения между режимами опроса и прерываний.
Зеленый светодиод	PD13	Выход Push-Pull, Low speed	Выход MCU	Индикатор VD6 «РАБОТА». Сигнализирует об открытии замка (свечение в течение 3 с).
Желтый светодиод	PD14	Выход Push-Pull, Low speed	Выход MCU	Секция YARC двухцветного индикатора VD7 «РЕЖИМ». Одиночная вспышка при верном символе кода.
Красный светодиод	PD15	Выход Push-Pull, Low speed	Выход MCU	Секция YCRA двухцветного индикатора VD7 «РЕЖИМ». Вспышка при ошибке ввода, мигание при тревоге 5 с.

Контроллер USART6 подключен к системной шине APB2. В соответствии с настройками тактового дерева микроконтроллера тактирование периферии APB2 осуществляется частотой 84 МГц (или 90 МГц в зависимости от конфигурации PLL). Модуль USART настраивается на формат кадра: 8 бит данных, без контроля четности (None Parity), 1 стоп-бит, скорость передачи 9600 бод (бит/с).

4. Архитектурное описание программы и алгоритмов

Встроенное программное обеспечение организовано по трехуровневой модульной схеме:

- **Аппаратно-зависимый уровень (драйверы):** модуль uart_driver (обслуживание приемопередатчика USART6 в режимах Polling и Interrupt), модуль ring_buffer (организация кольцевой очереди FIFO), модуль button (опрос кнопки с защитой от дребезга) и модуль led (неблокирующее управление светодиодной индикацией).
- **Прикладной логический уровень:** конечный автомат lock_fsm, инкапсулирующий состояния ввода, проверки, смены пароля и тревожной блокировки.
- **Уровень диспетчеризации:** бесконечный кооперативный цикл в файле main.c.

4.1. Диспетчеризация и принципы кооперативной многозадачности

Главный цикл программы функционирует в строго неблокирующем режиме без использования задержек HAL_Delay(). На каждом шаге итерации выполняются следующие этапы:

1. Считывание текущей системной миллисекундной метки HAL_GetTick().
2. Опрос драйвера кнопки: при фиксации устойчивого отпускания кнопки формируется событие переключения режима. Происходит вызов uart_driver_set_mode(), инвертирующий текущий режим работы с выводом текстового уведомления в UART, и сброс состояния автомата замка вызовом lock_fsm_reset().
3. Неблокирующее чтение символа из UART: вызов uart_driver_read_char(&rx_char). Если символ поступил (функция возвращает истину), он немедленно передается обработчику прикладной логики lock_fsm_process_char(rx_char). Если данных нет, цикл не задерживается.
4. Обновление прикладного автомата lock_fsm_update(now): контроль 5-секундного таймаута неактивности ввода и таймеров открытого/заблокированного состояний.
5. Обновление светодиодной индикации led_update(now): отработка неблокирующих вспышек и циклического мигания.

```
/* Листинг 4.1. Диспетчерский цикл кооперативной многозадачности (main.c) */
while (1) {
    uint32_t now = HAL_GetTick();

    /* 1. Опрос аппаратной кнопки стенда PC15 для смены режима */
    ButtonEvent btn_ev = button_update(now);
    if (btn_ev != BTN_EVENT_NONE) {
        if (uart_driver_get_mode() == UART_MODE_POLLING) {
            uart_driver_set_mode(UART_MODE_INTERRUPT);
            uart_driver_write_string("\r\n[Mode switched: INTERRUPTS ON]\r\n");
        } else {
            uart_driver_set_mode(UART_MODE_POLLING);
            uart_driver_write_string("\r\n[Mode switched: INTERRUPTS OFF (Polling)]\r\n");
        }
        lock_fsm_reset(); /* Сброс ввода согласно ТЗ */
    }
```

```
    /* 2. Неблокирующий приём символа из UART */
    char rx_char;
    if (uart_driver_read_char(&rx_char)) {
        lock_fsm_process_char(rx_char);
    }

    /* 3. Фоновое обновление автомата замка и анимаций светодиодов */
    lock_fsm_update(now);
    led_update(now);
}
```

```
/* Листинг 4.1. Диспетчерский цикл кооперативной многозадачности (main.c) */
while (1) {
    uint32_t now = HAL_GetTick();

    /* 1. Опрос аппаратной кнопки стенда PC15 для смены режима */
    ButtonEvent btn_ev = button_update(now);
    if (btn_ev != BTN_EVENT_NONE) {
        if (uart_driver_get_mode() == UART_MODE_POLLING) {
            uart_driver_set_mode(UART_MODE_INTERRUPT);
            uart_driver_write_string("\r\n[Mode switched: INTERRUPTS ON]\r\n");
        } else {
            uart_driver_set_mode(UART_MODE_POLLING);
            uart_driver_write_string("\r\n[Mode switched: INTERRUPTS OFF (Polling)]\r\n");
        }
        lock_fsm_reset(); /* Сброс ввода согласно ТЗ */
    }

    /* 2. Неблокирующий приём символа из UART */
    char rx_char;
    if (uart_driver_read_char(&rx_char)) {
        lock_fsm_process_char(rx_char);
    }

    /* 3. Фоновое обновление автомата замка и анимаций светодиодов */
    lock_fsm_update(now);
    led_update(now);
}
```

4.2. Архитектура драйвера UART и реализация кольцевого буфера FIFO

Для гарантированного сохранения входящего потока байтов в режиме прерываний разработан кольцевой буфер (Ring Buffer) размером 128 байт. Структура буфера содержит массив фиксированной длины и две беззнаковые целочисленные переменные-указателя, объявленные со спецификатором `volatile`:

- `head` — индекс головы буфера, куда обработчик прерывания записывает очередной принятый байт;
- `tail` — индекс хвоста буфера, откуда основной цикл программы вычитывает данные.

Буфер считается пустым при равенстве индексов (`head == tail`). Добавление элемента инкрементирует индекс `head` по модулю размера буфера; если следующий индекс совпадает с `tail`, фиксируется состояние переполнения, и новый байт отбрасывается без разрушения уже накопленных данных. Чтение элемента извлекает байт по индексу `tail` и сдвигает его по модулю размера очереди. Применение атомарных побайтовых операций чтения и записи для однобайтовых индексов исключает состояние гонки между прерыванием и главным циклом без необходимости блокировки глобальных прерываний.

4.3. Режимы работы драйвера UART: опрос и прерывания

Драйвер последовательного порта поддерживает динамическое переключение между двумя режимами:

- **Режим опроса (UART_MODE_POLLING):** прерывания от USART6 в блоке NVIC запрещены (NVIC_DisableIRQ(USART6_IRQn)), а бит разрешения прерывания по приему RXNEIE в регистре CR1 сброшен в 0. При вызове функции uart_driver_read_char() выполняется прямая проверка бита статуса RXNE в регистре SR: если бит равен единице, байт вычитывается из регистра данных DR (что автоматически аппаратно сбрасывает флаг RXNE) и возвращается вызывающей программе; если бит равен нулю, функция мгновенно возвращает false.
- **Режим прерываний (UART_MODE_INTERRUPT):** в регистре CR1 устанавливается бит RXNEIE, а линия прерывания активируется в контроллере NVIC (NVIC_EnableIRQ(USART6_IRQn)). При аппаратном приеме очередного байта микроконтроллер приостанавливает выполнение основного цикла и передает управление функции USART6_IRQHandler(), которая вызывает uart_driver_irq_handler(). Обработчик считывает байт из регистра DR, помещает его в кольцевой буфер и анализирует флаги аппаратных ошибок переполнения кадра (Overrun Error ORE), шума (Noise Error NE) и ошибки формата кадра (Framing Error FE), производя их программный сброс для исключения зависания интерфейса. Функция uart_driver_read_char() при этом читает байты исключительно из кольцевого буфера.

```
/* Листинг 4.2. Обработчик прерываний USART6 и вычитка из FIFO (uart_driver.c) */
void uart_driver_irq_handler(void) {
    if (s_huart == NULL) return;

    /* Проверка флага RXNE (регистр приёмника не пуст) */
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_RXNE) != RESET &&
        __HAL_UART_GET_IT_SOURCE(s_huart, UART_IT_RXNE) != RESET) {
        uint8_t received_byte = (uint8_t)(s_huart->Instance->DR & 0xFF);
        ring_buffer_put(&s_rx_ring, received_byte); /* Запись в FIFO */
    }

    /* Аппаратный сброс флага ошибки переполнения ORE (чтение SR затем DR) */
    if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_ORE) != RESET) {
        __HAL_UART_CLEAR_OREFLAG(s_huart);
    }
}

bool uart_driver_read_char(char *out_char) {
    if (s_mode == UART_MODE_INTERRUPT) {
        uint8_t byte;
        if (ring_buffer_get(&s_rx_ring, &byte)) {
            *out_char = (char)byte;
            return true;
        }
    } else {
        /* Polling: прямой опрос регистра статуса SR */
        if (__HAL_UART_GET_FLAG(s_huart, UART_FLAG_RXNE) == SET) {
            *out_char = (char)(s_huart->Instance->DR & 0xFF);
            return true;
        }
    }
    return false;
}
```

4.4. Логика конечного автомата кодового замка

Конечный автомат кодового замка оперирует пятью состояниями:

- **STATE_INPUT_CODE (Обычный ввод пароля):** эхо-вывод каждого символа в консоль. При получении символа + автомат очищает буфер временного пароля и переходит в режим ввода нового кода. В остальных случаях символ сопоставляется с эталоном: при совпадении выдается желтая вспышка (200 мс) и инкрементируется индекс; по достижении 8 правильных символов замок переходит в открытое состояние. При несовпадении выдается красная вспышка (300 мс), индекс обнуляется, а при 3 ошибках активируется блокировка тревоги. Если пауза между символами превышает 5 секунд, ввод сбрасывается по таймауту.
- **STATE_UNLOCKED (Замок открыт):** горит зеленый светодиод, в терминал отправлено сервисное сообщение об успешном доступе, нажатия клавиш и прием символов блокируются на 3 секунды. По окончании интервала зеленый светодиод выключается, и автомат возвращается в состояние ожидания ввода.
- **STATE_BLOCKED_ALARM (Блокировка тревоги):** красный светодиод циклически мигает с полупериодом 200 мс в течение 5 секунд, ввод заблокирован. По истечении таймера тревоги автомат возвращается в состояние ожидания ввода с обнуленным счетчиком ошибок.
- **STATE_NEW_CODE_INPUT (Ввод нового пароля):** принимаемые символы накапливаются во временном строковом массиве. Завершение ввода фиксируется при получении символа перевода строки (Enter) или по достижении предельной длины 8 символов, после чего автомат запрашивает подтверждение.

```
/* Листинг 4.3. Посимвольный разбор и логика открытия/блокировки (lock_fsm.c) */
void lock_fsm_process_char(char c) {
    if (s_state == STATE_UNLOCKED || s_state == STATE_BLOCKED_ALARM) return;
    uart_driver_write_char(c); /* Эхо-вывод */
    uint32_t now = HAL_GetTick();

    switch (s_state) {
        case STATE_INPUT_CODE:
            if (c == '+') { /* Режим смены пароля */
                s_state = STATE_NEW_CODE_INPUT;
                s_temp_code_len = 0;
                uart_driver_write_string("\r\n[Enter new 8-char code]:\r\n> ");
                return;
            }
            if (c == '\r' || c == '\n') return;

            /* Сравнение с активным 8-символьным паролем */
            if (char_match(c, s_active_code[s_input_idx])) {
                s_input_idx++;
                led_trigger_flash(LED_YELLOW, FLASH_YELLOW_MS); /* Вспышка жёлтого */
                if (s_input_idx >= CODE_MAX_LEN) {
                    /* Пароль совпал: открытие замка */
                    s_input_idx = 0; s_error_count = 0;
                    led_set(LED_GREEN, LED_STATE_ON); /* Зеленый на 3 сек */
                    s_state = STATE_UNLOCKED;
                    s_state_end_tick = now + UNLOCKED_TIME_MS;
                    uart_driver_write_string("\r\n*** ACCESS GRANTED (LOCK OPEN) ***\r\n");
                }
            } else {
                /* Ошибка: вспышка красного и сброс индекса */
                s_input_idx = 0; s_error_count++;
                led_trigger_flash(LED_RED, FLASH_RED_MS);
                uart_driver_write_string("\r\n[Wrong character! Reset to start]\r\n> ");
                if (s_error_count >= 3) { /* 3 ошибки -> блокировка на 5 сек */
```

```
                    s_state = STATE_BLOCKED_ALARM;
                    s_state_end_tick = now + ALARM_TIME_MS;
                    led_start_blink(LED_RED, 200, ALARM_TIME_MS);
                    uart_driver_write_string("\r\n!!! 3 WRONG ATTEMPTS! SYSTEM LOCKED
!!!\r\n");
                }
            }
            break;
    }
}
```

```
/* Листинг 4.3. Посимвольный разбор и логика открытия/блокировки (lock_fsm.c) */
void lock_fsm_process_char(char c) {
    if (s_state == STATE_UNLOCKED || s_state == STATE_BLOCKED_ALARM) return;
    uart_driver_write_char(c); /* Эхо-вывод */
    uint32_t now = HAL_GetTick();

    switch (s_state) {
        case STATE_INPUT_CODE:
            if (c == '+') { /* Режим смены пароля */
                s_state = STATE_NEW_CODE_INPUT;
                s_temp_code_len = 0;
                uart_driver_write_string("\r\n[Enter new 8-char code]:\r\n> ");
                return;
            }
            if (c == '\r' || c == '\n') return;

            /* Сравнение с активным 8-символьным паролем */
            if (char_match(c, s_active_code[s_input_idx])) {
                s_input_idx++;
                led_trigger_flash(LED_YELLOW, FLASH_YELLOW_MS); /* Вспышка жёлтого */
                if (s_input_idx >= CODE_MAX_LEN) {
                    /* Пароль совпал: открытие замка */
                    s_input_idx = 0; s_error_count = 0;
                    led_set(LED_GREEN, LED_STATE_ON); /* Зеленый на 3 сек */
                    s_state = STATE_UNLOCKED;
                    s_state_end_tick = now + UNLOCKED_TIME_MS;
                    uart_driver_write_string("\r\n*** ACCESS GRANTED (LOCK OPEN) ***\r\n");
                }
            } else {
                /* Ошибка: вспышка красного и сброс индекса */
                s_input_idx = 0; s_error_count++;
                led_trigger_flash(LED_RED, FLASH_RED_MS);
                uart_driver_write_string("\r\n[Wrong character! Reset to start]\r\n> ");
                if (s_error_count >= 3) { /* 3 ошибки -> блокировка на 5 сек */
                    s_state = STATE_BLOCKED_ALARM;
                    s_state_end_tick = now + ALARM_TIME_MS;
                    led_start_blink(LED_RED, 200, ALARM_TIME_MS);
                    uart_driver_write_string("\r\n!!! 3 WRONG ATTEMPTS! SYSTEM LOCKED
!!!\r\n");
                }
            }
            break;
    }
}
```

- **STATE_NEW_CODE_CONFIRM (Подтверждение смены):** система ожидает ввода символа подтверждения. При получении y или Y временная строка копируется в рабочий массив активного пароля, и замок возвращается в исходный режим с новым кодом; при получении любого иного символа смена пароля отменяется.

5. Анализ регистров USART в микроконтроллерах STM32F4

5.1. Архитектура и назначение регистров периферийного блока USART

В микроконтроллерах семейства STM32F4 аппаратный модуль USART управляется группой 32-битных регистров, подключенных к системным шинам APB1 или APB2:

Таблица 2 — Управляющие и информационные регистры периферийного блока USART в STM32F4

Регистр	Смещение	Название и назначение	Ключевые битовые поля
USART_SR	0x00	Регистр состояния (Status Register). Отображает текущие аппаратные флаги передачи, приема и ошибок интерфейса.	TXE (передающий регистр пуст), TC (передача завершена), RXNE (приемный регистр не пуст), ORE (ошибка переполнения приемника), NE (шум на линии), FE (ошибка формата кадра), PE (ошибка четности).
USART_DR	0x04	Регистр данных (Data Register). Служит для записи передаваемого или чтения принятого байта (младшие 9 бит).	DR[8:0] — физически разделен на буфер передатчика (TDR) и буфер приемника (RDR). Чтение извлекает данные из RDR, запись помещает данные в TDR.
USART_BRR	0x08	Регистр скорости передачи (Baud Rate Register). Задает коэффициент деления частоты системной шины для формирования бодрейта.	DIV_Mantissa[11:0] — целая часть коэффициента деления; DIV_Fraction[3:0] — дробная часть коэффициента деления.
USART_CR1	0x0C	Регистр управления 1 (Control Register 1). Включает приемопередатчик, задает длину слова, контроль четности и прерывания.	UE (включение USART), M (длина слова 8/9 бит), PCE (контроль четности), TXEIE (прерывание по пустому TXE), TCIE (прерывание по завершению передачи), RXNEIE (прерывание по приему байта RXNE), TE (разрешение передатчика), RE (разрешение приемника).
USART_CR2	0x10	Регистр управления 2 (Control Register 2). Настраивает количество	STOP[1:0] (количество стоп-битов: 00 — 1 стоп-бит, 01 — 0.5, 10 — 2 стоп-бита, 11 — 1.5 стоп-бита),

Регистр	Смещение	Название и назначение	Ключевые битовые поля
		стоп-битов, синхронный режим и аппаратный адрес.	CLKEN (синхронный тактовый сигнал).
USART_CR3	0x14	Регистр управления 3 (Control Register 3). Настраивает аппаратное управление потоком (CTS/RTS), полудуплекс и режимы DMA.	CTSE / RTSE (аппаратный контроль потока), DMAT / DMAR (разрешение каналов DMA на передачу и прием), EIE (прерывание по ошибкам).

5.2. Анализ механизмов обработки прерываний и сброса флагов

Флаг RXNE (Read Data Register Not Empty) устанавливается аппаратно в единицу, когда сдвиговый регистр приемника полностью принял стоп-бит кадра и перенес сформированный байт в регистр USART_DR. Если в регистре USART_CR1 установлен бит RXNEIE (RXNE Interrupt Enable), контроллер USART генерирует активный сигнал запроса прерывания на соответствующую линию векторного контроллера прерываний NVIC (для USART6 — вектор USART6_IRQn, номер позиции 71).

Сброс флага RXNE выполняется автоматически программным чтением регистра USART_DR, что исключает необходимость ручного сброса бита в регистре состояния. Однако при возникновении ошибки переполнения (Overrun Error, ORE), когда очередной байт принят до того, как предыдущий был вычитан из DR, интерфейс может заблокировать дальнейший прием. Для гарантированного восстановления работоспособности приемника процедура обработки ошибок требует последовательного чтения регистра SR с последующим чтением регистра DR, либо использования специализированных макросов библиотеки (таких как __HAL_UART_CLEAR_OREFLAG).

6. Методика и результаты экспериментальной верификации

Верификация разработанного программного обеспечения проводилась на лабораторном стенде SDK-1.1M при подключении к инструментальному персональному компьютеру через терминальную программу с параметрами порта: скорость 9600 бод, 8 бит данных, без контроля четности, 1 стоп-бит, без аппаратного управления потоком.

Таблица 3 — Протокол тестирования программного комплекса «Кодовый замок по UART»

№	Тестовый сценарий	Входное воздействие	Ожидаемая реакция системы	Фактический результат на стенде SDK-1.1M
1	Проверка эхо-вывода и приема в	Ввод произвольных символов в терминале при активном режиме Polling	Каждый введенный символ немедленно возвращается в терминал и	Символы мгновенно отображаются в окне терминала без

№	Тестовый сценарий	Входное воздействие	Ожидаемая реакция системы	Фактический результат на стенде SDK-1.1M
	режиме опроса (Polling)		отображается на экране	задержек и пропусков
2	Полный правильный ввод пароля (открытие замка)	Ввод активной кодовой комбинации: «1234abcd» (с проверкой нечувствительности к регистру «1234ABCD»)	На каждый символ — желтая вспышка 200 мс. По завершении — включение зеленого LED на 3 с, сообщение об открытии, временная блокировка ввода	Желтый LED четко отработал 8 вспышек; зеленый LED загорелся ровно на 3 с; в терминал выведена строка «*** ACCESS GRANTED (LOCK OPEN) ***»
3	Одиночная ошибка ввода символа	Ввод неверного символа (например, «x» вместо «1» или на промежуточной позиции)	Одиночная вспышка красного LED (300 мс), сброс позиции ввода в начало, вывод уведомления об ошибке в терминал	Красный LED кратковременно вспыхнул на 300 мс; ввод сброшен в начало; в терминал выведено «[Wrong character! Reset to start]»
4	Трехкратная ошибка ввода подряд (режим тревоги)	Три подряд ошибочных попытки ввода символов последовательности	Включение аварийного режима: непрерывное мигание красного LED в течение 5 с, вывод предупреждения, блокировка приема	Красный LED циклически мигал с полупериодом 200 мс ровно 5 секунд; ввод символов игнорировался; после тревоги система готова к новому вводу
5	Таймаут неполного ввода пароля	Ввод начальных символов пароля («1234»), после чего пауза бездействия более 5 секунд	Вспышка красного LED на 300 мс, сброс ввода в начало, отправка сообщения об истечении таймаута в терминал	Через 5 секунд после последнего ввода красный LED вспыхнул на 300 мс; в терминал выведено «[Input timeout! Reset to

№	Тестовый сценарий	Входное воздействие	Ожидаемая реакция системы	Фактический результат на стенде SDK-1.1M
				start]»; позиция сброшена
6	Динамическая смена пароля замка	Ввод символа «+», затем нового пароля «pass2026», нажатие Enter и подтверждение символом «y»	Переход в режим смены пароля, запрос подтверждения, сохранение нового пароля и успешное открытие по новой комбинации	Замок запросил подтверждение «Make this code active? (y/n): »; после ввода «y» новый пароль активирован; открытие замка проверено новым кодом
7	Переключение между режимами Polling и Interrupts по кнопке стенда	Нажатие тактовой кнопки боковой панели PC15 во время работы	Инверсия текущего режима работы UART, вывод уведомления о смене режима в консоль, сброс текущего состояния ввода замка	Режим мгновенно переключился с выводом сообщения «[Mode switched: INTERRUPTS ON]»; прием через кольцевой FIFO-буфер функционирует стабильно

7. Выводы

В ходе выполнения лабораторной работы были всесторонне изучены принципы организации, схемотехники и программирования последовательного интерфейса UART и системы векторных прерываний NVIC на примере микроконтроллера STM32F427VIT6 учебного стенда SDK-1.1M:

1. Спроектирован и реализован универсальный двухрежимный драйвер последовательного интерфейса UART (USART6) со скоростью передачи 9600 бит/с: режим неблокирующего опроса аппаратных флагов регистра SR (Polling) и режим асинхронной обработки прерываний по приему RXNE (Interrupt).
2. Для режима прерываний разработан надежный кольцевой FIFO-буфер емкостью 128 байт, гарантирующий защиту от потери данных при кратковременной занятости процессора и аппаратных задержках главного цикла, а также реализована обработка ошибок переполнения кадра (ORE), шума (NE) и формата (FE).

3. Модернизирован прикладной конечный автомат «Кодовый замок» (Вариант 5) для управления через терминальный интерфейс: реализован регистронезависимый ввод 8-символьного кода, светодиодная индикация корректных шагов, ошибок и аварийной блокировки при трех ошибках подряд, 5-секундный таймаут бездействия и процедура защищенного изменения активного пароля через специальную командную последовательность.
4. Обеспечено гибкое переключение режимов работы UART (Polling / Interrupt) по нажатию аппаратной кнопки стенда PC15 с автоматическим сбросом параметров ввода и трансляцией сервисных сообщений в терминал.
5. Экспериментальное тестирование подтвердило стабильность передачи данных, высокую отзывчивость системы и полное соответствие разработанного решения всем требованиям технического задания.